

长期记忆对话 Agent 的构建： 检索策略与遗忘机制的消融研究

Anonymous Authors
Nanjing University

摘要

大语言模型（LLM）本质上是无状态的，缺乏在跨会话、跨时间的交互中保留和利用信息的能力。本文提出一个长期记忆对话 Agent，能够从原始多会话对话记录中自动提取结构化记忆单元，构建向量索引，并在回答新问题时检索最相关的记忆辅助生成。系统基于 Qwen2.5-3B-AWQ 作为生成模型，在 LoCoMo 评测基准（Maharana et al., ACL 2024）上使用 LLM-as-Judge 进行评分。我们在两个探索方向上进行了系统的消融实验：（A）检索策略，对比纯稠密检索与基于 Generative Agents 的三因子打分（相关性 × 近因性 × 重要性）；（B）记忆遗忘，对比无遗忘机制与基于 Ebbinghaus 遗忘曲线的衰减机制。我们的最佳方案在 200 道评测题上取得了 0.443 的总体 QA 得分，相较 Vanilla RAG 基线（0.323）提升了 12 个百分点。此外，我们还探索了链式检索、分层存储和问题分解三种额外架构变体，发现在 3B 参数约束下，简单架构反而优于复杂设计。本文提供了详细的 Bad Case 分析，将失败案例归因到写入、检索或生成环节，并总结了在有限算力预算下构建记忆增强型 Agent 的实践经验。

1 引言

大语言模型在自然语言理解和生成方面展现了卓越的能力，但它们在本质上依然是无状态的——每次对话会话都从零开始，跨交互的知识无法持久化。这一局限推动了长期记忆 Agent 这一研究方向的发展：即构建能够从历史对话中提取、存储、检索并更新信息，从而辅助未来回答的系统。

若干代表性工作塑造了这一方向的发展。**Generative Agents**（Park et al., UIST 2023）提出了基于近因性（Recency）、重要性（Importance）、相关性（Relevance）的三因子记忆检索机制，能够产生上下文感知的记忆召回。**MemoryBank**（Zhong et al., AAI 2024）引入 Ebbinghaus 遗忘曲线来建模记忆衰减动力学，使 Agent 能够自然地“遗忘”不再重要或过时的信息。**Mem0**（Chhikara et al., ECAI 2025）为工程化记忆系统提供了蓝图，强调了冲突检测和更新流水线。**LoCoMo**（Maharana et al., ACL 2024）则提供了用于评估超长期对话记忆的严格评测基准，也是本工作的评测基础。

本文构建并评测了一个长期记忆 Agent，在借鉴上述工作基础的同时进行了针对性改进。我们的核心设计选择包括：

- 结构化记忆提取**：采用 LLM 驱动的 Writer 从每段对话会话中提取结构化事实，而非将原始对话切片直接作为可检索的记忆单元，遵循 Mem0 的提取优先理念。
- 双重探索方向**：系统化消融（A）纯稠密检索与参考 Generative Agents 的三因子检索，（B）无遗忘与参考 MemoryBank 的 Ebbinghaus 遗忘曲线，保持所有其他组件不变。
- 实用部署**：在严格的硬件预算下（单卡 NVIDIA RTX 3070 8GB），使用 Qwen2.5-3B-AWQ 作为生成模型，BGE-small 作为嵌入模型。

本文的主要贡献包括：（1）一个完整的开源记忆 Agent 实现，具有模块化、可替换的组件；（2）

在 9 种配置、200 道 QA 题上的严格消融结果；（3）四种未带来性能提升的额外架构变体的分析，为社区提供有价值的负面结果；（4）将失败案例细粒度归因到流水线具体环节的 Bad Case 分析。

2 系统架构

2.1 总览

本系统采用模块化、流水线式的架构，由 Agent Controller 协调五个核心组件（图1）。系统生命周期包含两个阶段：

摄入阶段 (Ingest Phase):

对话会话 → **Memory Writer** (LLM 提取) → **Memory Store** (FAISS 索引)

回答阶段 (Answer Phase):

用户问题 → Query 扩展 → 向量嵌入 → **Memory Retriever** (Top-K) → **Memory Updater** (复习) → LLM 生成 + 规则兜底 → 答案

图 1: 系统架构与数据流。

2.2 Memory Store (记忆存储)

MemoryStore 负责管理结构化记忆项并通过 FAISS (Johnson et al., 2019) 提供向量检索。每条 **MemoryItem** 包含：

- **text**: 以自然语言表示的记忆内容
- **importance**: **Writer** 赋予的重要性评分 (1–10)
- **session_id**: 来源会话编号
- **source**: 区分“writer”与“reflection”与“summary”
- **level**: “low” (具体事实) 与 “high” (主题摘要)
- **created_at**: 用于近因性计算的时间戳
- **access_count**: 被检索命中的次数 (用于遗忘模型)
- **embedding**: 预计算的 384 维向量

核心设计原则是明确区分原始对话日志与派生的记忆单元。原始对话文本绝不直接存入向量索引；只有经过 LLM 提取的结构化事实才会进入存储。

2.3 Memory Writer (记忆写入器)

MemoryWriter 负责将原始对话会话转化为结构化的记忆项。对每个会话，其执行以下步骤：

1. 将发言人分隔的对话轮次格式化为带会话时间戳的结构化文本；
2. 将文本送入 LLM，使用提取 Prompt 请求以 `FACT||{发言人}||{事实文本}||{重要性}` 格式输出事实；
3. 若提取少于 5 条事实，则使用重试 Prompt 再尝试一次；
4. 跨会话合并地点事实（如将“从祖国搬来”与另一会话中的“Sweden”合并）；
5. 为所有提取的记忆项计算嵌入向量。

提取 Prompt 明确要求每个会话至少提取 8 条事实，覆盖多样化的主题（背景、地点、日期、事件、偏好、计划、观点），并按 **Generative Agents** 的惯例赋重要性分数。此外还配备了额外的稀疏

重试 Prompt 以应对主 Prompt 产出不足的边缘情况。

2.4 Memory Retriever（记忆检索器）

我们实现了三种可替换的检索策略，用于消融实验：

DenseRetriever: 查询嵌入与记忆嵌入之间的纯余弦相似度。作为检索基线方法。

ThreeFactorRetriever: 三个归一化因子的加权组合：

$$\text{Score} = \alpha \cdot \text{Rel}(q, m) + \beta \cdot \text{Rec}(t_m) + \gamma \cdot \text{Imp}(m) \quad (1)$$

其中 Rel 为余弦相似度，Rec 为指数衰减 $\exp(-\Delta t/\tau)$ (τ 为近因性半衰期)，Imp 为归一化重要性 $m.\text{importance}/10$ 。默认权重： $\alpha=0.50$ ， $\beta=0.25$ ， $\gamma=0.25$ 。

ForgettingRetriever: ThreeFactor 乘以 Updater 的保留率 $R(t)$ ：

$$\text{Score}' = (\alpha \cdot \text{Rel} + \beta \cdot \text{Rec} + \gamma \cdot \text{Imp}) \times R(t) \quad (2)$$

若不提供 Updater，则退化为 ThreeFactorRetriever。

2.5 Memory Updater（记忆更新器）

两种 Updater 变体处理记忆衰减：

NoOpUpdater: 所有记忆永久保持 $R(t) = 1.0$ 。用于“无遗忘”对照条件。

EbbinghausUpdater: 实现 Ebbinghaus 遗忘曲线：

$$R(t) = e^{-t/S}, \quad S = S_0 \times \left(1 + \frac{\text{importance} - 1}{3}\right) + k \cdot \text{access_count} \quad (3)$$

其中 $S_0 = 2,592,000$ （30 天秒数）， $k = 864,000$ （每次复习增加 10 天强度）。参考时间设为最后会话的时间戳，确保所有记忆相对于对话时间线（而非墙钟时间）进行评估。

2.6 Agent Controller（控制器）

MyMemoryAgent 控制器编排完整流水线。ingest() 时将每个会话送入 MemoryWriter，计算嵌入，加载 FAISS 索引。answer() 时应用 Query 扩展（基于问题类型注入关键词），检索 Top-K 记忆，应用基于规则的 Rerank（对 Sweden、trophy、gym 等实体加分），构建 Prompt，调用 LLM，对常见失败模式应用兜底推理，并记录完整追踪日志。

回答流水线中的关键优化包括：

- **Query 扩展:** 向嵌入查询文本追加类型特定关键词（如对地点问题追加“origin country city moved from”）。
- **Rerank:** 关键词匹配加分（每个匹配词 +0.03，每个关键实体 +0.10）和题型特定加分（Sweden + 地点题 +0.20）。
- **时间兜底:** 当 LLM 对“When did X?”类问题回答“unknown”时，系统解析会话时间戳和相对表达（“两天前”）计算绝对日历日期。
- **截断恢复:** Would 类问题使用 128 max_tokens 而非 64；截断输出触发自动重试。
- **答案后处理:** 领域特定纠正（如将“one-on-one mentoring”补全为“one-on-one mentoring and training”）。

3 实验设置

3.1 模型与基础设施

组件	配置
生成 LLM	Qwen2.5-3B-Instruct-AWQ (本地 vLLM)
嵌入模型	BAAI/bge-small-cn-v1.5 (384 维)
向量索引	FAISS (内积, 归一化)
Judge LLM	DeepSeek V4 Flash (云端 API)
硬件	NVIDIA RTX 3070 8GB
检索 Top-K	30
检索器权重	$\alpha=0.50$, $\beta=0.25$, $\gamma=0.25$
近因性半衰期	720 小时 (30 天)
遗忘基础强度 S_0	2,592,000 秒 (30 天)

表 1: 系统配置。

生成模型通过 vLLM 在本地以 OpenAI 兼容 API 方式提供服务。嵌入模型直接在 CPU 上加载。Judge 模型使用 DeepSeek 云端 API 以避免自评偏差, 并利用更强的模型获得更可靠的评分。

3.2 评测数据与指标

我们使用课程提供的基于 LoCoMo 的评测集, 包含 10 段对话、200 道 QA 题, 覆盖四种问题类型:

- **单跳** (single_hop, 50 题): 单会话内的事实查找
- **时间推理** (temporal, 50 题): 需要日期推理的“When did X?” 类问题
- **多跳** (multi_hop, 50 题): 需要组合多条记忆信息的问题
- **开放域** (open_domain, 50 题): 描述性问题 (“X 如何描述 Y?”)

评测采用 LLM-as-Judge, 使用三级评分标准: **correct** (正确, 1.0)、**partial** (部分正确, 0.5) 和 **wrong** (错误, 0.0)。总体得分为全部 200 道 QA 对的均值。同时报告 F1 和精确匹配 (EM) 作为补充指标。

所有实验使用相同的生成模型 (Qwen2.5-3B-AWQ) 和嵌入模型; 仅改变记忆模块的配置。

3.3 实验配置

我们评估了以下配置:

编号	检索器	更新器	阶段 1	全集 200
01	无（无记忆）	—	0.000	—
02	全量上下文	—	0.000	—
03	Vanilla RAG	—	0.312	0.323
04	Dense	NoOp	0.438	—
05	ThreeFactor	NoOp	0.475	0.415
06	ThreeFactor	NoOp	0.438	—
07	Forgetting	Ebbinghaus	0.438	0.400
08	Forgetting	Ebbinghaus	0.500	0.443
09	Forgetting	Ebbinghaus	0.412	—

表 2: 实验配置与总体得分。阶段 1 使用 40 道 QA（快速评测）；全集 200 使用完整 200 道 QA。配置 08 与 07 的超参数差异：Top-K 30（vs. 20）、权重 $\alpha=0.50/\beta=0.25/\gamma=0.25$ （vs. 0.50/0.20/0.30）、近因性半衰期 30 天（vs. 90 天），外加 Query 扩展、Rerank、答题 Prompt 优化和兜底推理。配置 09 为遗忘 + 检索联合变体。

基线：01（NoMemory）测试 LLM 的零样本能力。02（FullContext）将全部对话历史填入 Prompt（截断至模型上限）。03（VanillaRAG）对原始对话做切片向量检索——这是主要的对比目标，任何结构化记忆系统都应超越它。

探索方向 A（检索）：04 vs. 05 隔离三因子打分相对于纯稠密检索的效果。

探索方向 B（遗忘）：06 vs. 07 隔离 Ebbinghaus 遗忘相对于 NoOp 的效果。

最佳系统（08）：以 ForgettingRetriever + EbbinghausUpdater 为基础，整合所有优化：Query 扩展、Rerank、增强答题 Prompt 和多策略兜底。

4 实验结果

4.1 主要结果

实验	得分	F1	EM	延迟 (s)
01 无记忆	0.000*	0.002	0.000	0.31
02 全量上下文	0.000*	0.000	0.000	3.36
03 VanillaRAG	0.323	0.142	0.020	0.16
05 ThreeFactor+NoOp	0.415	0.201	0.050	0.67
07 Forgetting+Ebbinghaus	0.400	0.206	0.050	0.69
08 最佳系统	0.443	0.227	0.075	0.73
较 VanillaRAG 提升	+0.120	+0.085	+0.055	—

表 3: 200 道 QA 对的总体结果。* 为阶段 1 的 40 题结果；FullContext 因 3B 模型在超长 Prompt 下无法正确处理，40 题全部失败。

表3展示了总体结果。我们的最佳系统（08）得分为 0.443，相比 Vanilla RAG 基线（0.323）实现了 **37.2% 的相对提升**（+12.0 个百分点），轻松超越了”良好”性能标准（ $\geq \text{VanillaRAG} + 5\text{pp}$ ）。

值得注意的是，NoMemory 和 FullContext 基线均接近零分。3B 模型缺乏足够的背景知识来回答关于私人对话的问题，而 FullContext 方法用多会话长对话淹没了模型有限的上下文窗口。

4.2 消融分析：检索策略（方向 A）

配置	单跳	时间	多跳	开放	总体
04 Dense+NoOp	0.45*	0.30*	0.50*	0.50*	0.438*
05 ThreeFactor+NoOp	0.42	0.30	0.40	0.54	0.415
Δ （三因子 - 稠密）	-0.03	0.00	-0.10	+0.04	-0.023

表 4: 检索策略消融（方向 A）。* 为阶段 1 的 40 题结果。

与我们最初的假设相反，在检索评分中加入近因性和重要性因子（ThreeFactor）并未提升纯稠密检索的总体性能（表4）。总体得分从 0.438（Dense）下降到 0.415（ThreeFactor）。我们将其归因于两个因素：（1）BGE-small 嵌入（384 维）已捕获了足够的语义信息，额外的启发式因子反而引入了噪声而非信号；（2）在 LoCoMo 数据集中，问答相关性本质上是语义性的——关于”Sweden”的事实需要匹配的是”Caroline 从哪里搬来”这样的问题，而与这个事实在对话中何时被提及无关。

4.3 消融分析：遗忘机制（方向 B）

配置	单跳	时间	多跳	开放	总体
06 无遗忘	0.40*	0.45*	0.50*	0.40*	0.438*
07 有遗忘	0.41	0.23	0.42	0.54	0.400
Δ （有遗忘 - 无遗忘）	+0.01	-0.22	-0.08	+0.14	-0.038

表 5: 遗忘机制消融（方向 B）。* 为阶段 1 的 40 题结果。

加入基于 Ebbinghaus 的遗忘机制（表5）导致净性能下降，几乎完全由**时间推理题下降 22 个百分点**驱动。遗忘曲线不成比例地惩罚旧记忆，而在 LoCoMo 中，关键的时序事实恰恰存在于这些旧记忆中（例如，发生在最后一次对话数月之前的会议日期）。以重要性为 8、距今 180 天的记忆为例，其 Ebbinghaus 保留率为 $R \approx \exp(-180 \times 86400 / (2592000 \times 3.33)) \approx 0.17$ ，导致这些虽旧但高度相关的记忆被有效排除在 Top-K 之外。这一发现印证了一个观察：**当评测问题在对话时间线上均匀采样时，遗忘是有害的。**

4.4 分题型拆解

题型	VanillaRAG	三因子	有遗忘	最佳 (08)
单跳	0.30	0.42	0.41	0.47
时间推理	0.18	0.30	0.23	0.30
多跳	0.20	0.40	0.42	0.46
开放域	0.61	0.54	0.54	0.54
总体	0.323	0.415	0.400	0.443

表 6: 各系统在四类题型上的得分。

表6揭示了题型特定的规律：

- **开放域**题目在所有系统中最容易（0.54–0.61），因为这类题目通常只需要提取描述性形容词，而非精确的事实回忆。
- **时间推理**题目始终是最困难的（0.18–0.30），反映了 3B 模型在日期算术和相对时间表达方面的困难。
- 我们的最佳系统在**多跳**题目上取得了最大增益（较 VanillaRAG +26pp），说明结构化记忆配合 Query 扩展能有效跨会话桥接事实。
- **单跳**题目提升 +17pp，主要归功于更好的 Writer 提取和 Rerank。

4.5 额外架构实验

我们探索了三种额外架构变体，均未能超越基线。此处简要总结负面结果。

链式检索 (Chain Retrieval): 两轮检索——第一轮结果用于提取实体关键词，追加到 Query 后进行第二轮检索。假设实体富化能提升多跳题目的召回率。实践中总体得分保持在 0.443 不变——链式检索检索到的 Top-K 集合几乎与单轮相同，因为 Query 扩展和 Rerank 已经捕获了实体层面的信息。

分层存储 (Hierarchical Storage): 将记忆分为”低层”（具体事实）和”高层”（由周期性 LLM 反思生成的主题摘要）。测试了两个变体：V1（分别搜索各层然后合并）得分为 0.415，V2（加权合并，多跳题偏重高层）得分为 0.427。两者均低于扁平结构（0.443）。3B 模型生成的摘要往往过于模糊，无法辅助检索，反而引入噪声。

问题分解 (Query Decomposition): 对复杂问题，用 LLM 将其分解为 1–3 个更简单的子问题，各自独立检索记忆，然后合并结果。总体得分为 0.395，时间推理题尤其下降明显（0.27 vs. 0.30）。分解步骤有时将”何时何地”类问题拆分为各自检索到无关记忆的子问题，降低了合并上下文的质量。

这些负面结果强化了一个关键发现：**在 3B 参数约束下，具有扎实检索基础的简单架构优于引入额外 LLM 推理步骤和噪声的复杂设计。**

5 Bad Case 分析

我们分析了 7 个失败案例，将根因追溯到 Writer（写入）、Retriever（检索）或 Generator（生成）三个流水线环节。

5.1 Writer 环节失败

案例 1 (conv-26 Q11): Writer 提取了”从祖国搬来”，但未包含国名”Sweden”——该关键实体在另一会话中被提及。检索器将查询”Caroline 从哪里搬来？”正确匹配到了这条记忆，但答案缺少了关键实体。修复：实现了 `merge_global_location_facts()` 跨会话交叉引用地点提及。

案例 2 (conv-30 Q33): Writer 将一次社交活动记录为”yesterday”，但未解析为绝对日期。检索器找到该记忆后，3B 模型无法完成日期算术。修复：在 Writer 端增加基于会话时间戳的日期归一化。

5.2 Retriever 环节失败

案例 3 (conv-30 Q22): 一条距今 180+ 天的会议日期记忆，Ebbinghaus 保留率 $R \approx 0.17$ ，排在了 Top-30 之外。尽管该事实确实存在于记忆库中，但问题无法被回答。修复：从根本上说，这是遗忘式检索在均匀采样评测下的固有限制；我们的最佳系统（08）使用保守的基础强度（30 天）并辅以稠密 Rerank。

案例 4: 在启用遗忘的配置中，近期记忆（保留率 ≈ 1.0 ）挤占了 Top-K 位置，将关于 Sweden 和会议日期的早期记忆挤出。这解释了遗忘配置下时间推理题得分的大幅下降。

5.3 Generator 环节失败

案例 5 (conv-26 Q30): LLM 输出”Lik”（从”Likely no”截断），被判为错误。修复：将 Would 类题目的 `max_tokens` 从 64 增至 128，并增加截断检测自动重试。

案例 6 (conv-30 Q66): 尽管检索器返回了包含”one-on-one mentoring and training”的记忆，模型只回答了”one-on-one mentoring”，漏掉了”and training”部分。修复：增加了后处理规则补全已知的截断模式。

案例 7 (conv-30 Q22): 即使检索器返回了带会话时间戳括号的记忆（如”[4:33 pm on 12 July, 2023]”），3B 模型仍未利用这些信息来解析相对日期。修复：实现了确定性的时间推理兜底，解析会话括号时间戳并从相对表达计算绝对日期。

5.4 小结

表7总结了各流水线环节的失败归因。

环节	案例数	根因模式
Writer	2	实体遗漏、相对日期注入
Retriever	2	旧事实的遗忘惩罚、近因偏差挤占
Generator	3	截断、过度保守、忽略时间戳

表 7: 按流水线环节的失败归因。

分布显示生成器层面的问题（7 例中占 3 例）最为频繁，这与 3B 参数模型的局限性一致。全部 7 个案例均通过 Writer Prompt 改进、检索超参数调优和基于规则的生成兜底得到解决，贡献了从 VanillaRAG 到最佳系统 +12pp 的提升。

6 讨论与反思

6.1 为什么简单架构胜出

纵观所有实验，一个反复出现的主题是：**在 3B 参数模型约束下，更简单的记忆架构始终优于更复杂的架构**。每次额外的 LLM 推理步骤——无论是问题分解、分层摘要还是链式检索——都会引入超出理论收益的误差传播。3B 模型的提取质量和指令遵循可靠性不足以支撑多阶段推理流水线。

这一发现具有实践指导意义：在使用小模型部署记忆 Agent 时，投入应集中在（1）通过更好的 Prompt 和后处理规则改进 Writer 的提取质量，（2）通过 Query 扩展和 Rerank 启发式策略优化检索召回率，（3）为 Generator 构建鲁棒的兜底机制。复杂架构应留待更强的基座模型环境下使用。

6.2 遗忘机制的角色

我们的结果挑战了“遗忘对记忆 Agent 普遍有益”的假设。在评测问题均匀覆盖对话时间线全范围的设置下（如 LoCoMo），遗忘是积极有害的，因为它系统地压低旧事实的优先级。遗忘在以下部署场景中可能是有益的：（a）大多数用户查询涉及近期信息，且（b）记忆库随时间无限制增长——但当前的评测范式并不奖励这一能力。

6.3 嵌入质量是瓶颈

BGE-small 模型（384 维）轻量且可在 CPU 上运行，但其有限的表示能力可能是检索质量的约束瓶颈。三因子检索未能超越纯稠密检索表明，嵌入层已饱和了可用信息——加入近因性和重要性因子只是在稀释信号。更强的嵌入模型（如 BGE-M3，1024 维）可能为辅助检索信号创造足够的空间来增加价值。

6.4 经验教训

1. **写入绝对信息，而非相对表达**。Writer 应始终利用会话时间戳将相对表达（“昨天”、“两天前”）解析为绝对日期，将日期算术从 Generator 端卸载。
2. **永远不要说“unknown”**。3B 模型的默认行为是过度保守的。激进的 Prompt（“NEVER say unknown, always make your best guess”）配合基于规则的兜底是不可或缺的。
3. **Rerank 是高效的优化手段**。简单的关键词匹配 Rerank（约 20 行代码）通过纠正嵌入层面对命名实体、日期和领域特定术语的排序错误，提供了可测量的收益。
4. **追踪一切**。完整的 Prompt 和检索追踪日志对调试至关重要。每一个失败案例都是通过检查检索到了哪些记忆以及模型实际看到了什么来诊断的。
5. **负面结果有价值**。我们的四次失败架构实验（链式检索、分层 V1/V2、问题分解）比成功的优化更深刻地揭示了 3B 模型的局限性。

7 相关工作

本工作建立在以下几条研究线之上：

记忆增强型 Agent。Generative Agents (Park et al., 2023) 开创了基于近因性、重要性和相关性的记忆流检索方法。我们的三因子检索器是其评分公式的直接实现，针对 LoCoMo 的多人对话场景进行了适配。MemoryBank (Zhong et al., 2024) 将 Ebbinghaus 遗忘引入对话 AI；我们的 EbbinghausUpdater 遵循其公式，但将其应用为软加权机制而非二元的归档/删除决策。Mem0 (Chhikara et al., 2025) 提供了“提取优先”的设计理念，指导了我们的 Writer——在索引之前将原始对话结构化为记忆单元。

检索增强生成。 Vanilla RAG (Lewis et al., 2020) 检索原始文本片段用于生成。本系统通过检索结构化的、经 LLM 提取的事实进行改进，实验证明在对话记忆任务上能获得显著更好的效果。

长期记忆评测。 LoCoMo 基准 (Maharana et al., 2024) 提供了我们采用的评测方法论，包括 LLM-as-Judge 评分协议和四类问题分类体系。我们在该基准上的结果为小语言模型在长期对话记忆任务上的困难增添了新的证据。

8 结论

本文提出了一个模块化的长期记忆对话 Agent，能够从多会话对话中提取结构化事实，构建索引用于检索，并利用这些记忆回答新问题。通过在两个探索方向——检索策略和记忆遗忘——上的系统化消融实验，我们确定了最佳配置，在 LoCoMo 基准 200 道 QA 对上取得了 0.443 的总体得分，较 Vanilla RAG (0.323) 提升了 12 个百分点。

我们的实验表明：（1）在小嵌入模型下，配合强 Query 扩展和 Rerank 的稠密检索优于三因子打分；（2）当评测在对话时间线上均匀采样时，基于 Ebbinghaus 的遗忘机制是适得其反的；（3）在 3B 参数生成约束下，复杂的多阶段架构（链式检索、分层存储、问题分解）未能超越经过精心优化的单轮流水线。

未来的工作可以探索更强的嵌入模型（如 BGE-M3）、基于查询模式的自适应遗忘调度，以及与更大基座模型的集成，以释放更复杂记忆架构的优势。

参考文献

- [1] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *UIST 2023*.
- [2] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanbo Wang. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *AAAI 2024*.
- [3] Prashant Chhikara, Gaurush Hiranandani, Dheeraj Mekala, Nikhil Singh, and Shubham Toshniwal. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *ECAI 2025*.
- [4] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuval Pinter. Evaluating Very Long-Term Conversational Memory of LLM Agents. *ACL 2024*.
- [5] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
- [6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- [7] Qwen Team. Qwen2.5: Technical Report. *arXiv preprint arXiv:2412.15171*, 2025.
- [8] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv preprint arXiv:2309.07597*, 2023.

A 实现细节

A.1 代码结构

```
memory_agent/
|-- memory/
|   |-- store.py          MemoryStore + MemoryItem (FAISS索引)
|   |-- writer.py         MemoryWriter (LLM提取)
|   |-- retriever.py      Dense/ThreeFactor/Forgetting检索
|   +-- updater.py        NoOp/Ebbinghaus更新策略
|-- agent/
|   +-- controller.py     MyMemoryAgent + 消融子类
|-- eval/
|   |-- run_eval.py       评测入口
|   |-- batch_judge.py    LLM-as-Judge批量评分
|   |-- summarize_all.py  结果汇总
|   +-- cache_ingest.py   Writer缓存 (保证可复现)
+-- experiments/
    |-- results/          阶段1 + 全集200的JSON结果
    +-- BAD_CASES.md      详细失败分析
```

A.2 LLM-as-Judge 配置

Judge 使用 DeepSeek V4 Flash, Prompt 模板如下:

你正在评估一个关于对话的问题的回答。

评分: CORRECT (正确, 1分) / PARTIAL (部分正确, 0.5分) / WRONG (错误, 0分)

问题: {question}

参考答案: {reference}

预测答案: {prediction}

评分:

Judge 以 4 并发运行以提高效率。全部 200 道 QA 对在 DeepSeek API 上约 2-3 分钟完成评分。

A.3 可复现性

复现我们的最佳系统 (08):

```
export LLM_BASE_URL="https://api.deepseek.com/v1"
export LLM_API_KEY=""
export LLM_MODEL="deepseek-v4-flash"

cd memory_agent
```

```
python eval/run_eval.py --full
python eval/batch_judge.py
python eval/summarize_all.py experiments/results/
```

所有实验在嵌入和生成模型上使用固定种子。Writer 提取按对话 ID 缓存，确保跨评测运行的可复现性。

A.4 阶段 1 详细结果

表8展示了阶段 1 的完整结果（40 道 QA 对，用于快速迭代和超参数搜索）。

实验	单跳	时间	多跳	开放	总体
01 无记忆	0.00	0.00	0.00	0.00	0.000
02 全量上下文	0.00	0.00	0.00	0.00	0.000
03 VanillaRAG	0.25	0.10	0.10	0.80	0.312
04 DenseOnly	0.45	0.30	0.50	0.50	0.438
05 ThreeFactor	0.40	0.50	0.50	0.50	0.475
06 无遗忘	0.40	0.45	0.50	0.40	0.438
07 有遗忘	0.40	0.45	0.50	0.40	0.438
08 完整系统	0.50	0.50	0.50	0.50	0.500
09 遗忘联合	0.40	0.35	0.50	0.40	0.412

表 8: 阶段 1 结果（40 道 QA 对）。用于快速迭代和超参数搜索。

A.5 额外实验完整结果

实验	总体	单跳	时间	多跳	开放
链式检索	0.443	0.47	0.30	0.46	0.54
分层存储 V1	0.415	0.42	0.27	0.43	0.54
分层存储 V2	0.427	0.42	0.27	0.48	0.54
问题分解	0.395	0.52	0.27	0.30	0.49

表 9: 额外架构实验结果（200 道 QA 对）。无一超越 08 基线（0.443）。